

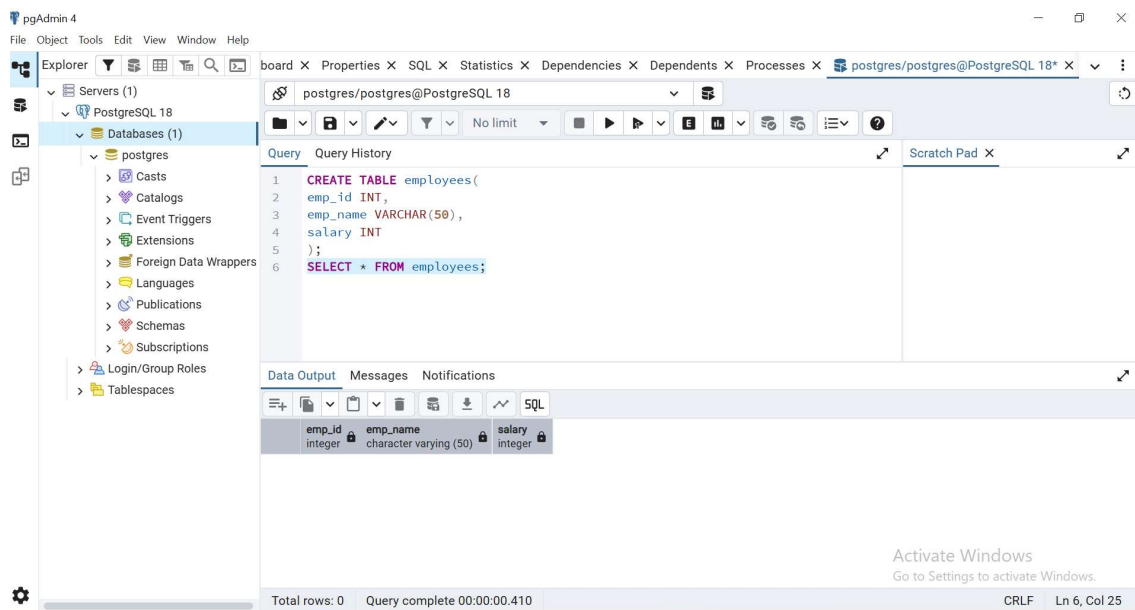
POSTGRESQL CASE FUNCTIONS

Scenario: Employee Salary Classification

Step 1: Create Table

- CREATE TABLE employees (
 - emp_id INT,
 - emp_name VARCHAR(50),
 - salary INT
-);
- SELECT * FROM employees;

STEP 2: Verified Data Output → Displays the employees table been created.



STEP 3: INSERT DATA

- INSERT INTO employees (emp_id,emp_name,salary)
- VALUES
- (101, 'Priya', 25000),
- (102, 'Jayaraj', 55000),
- (103, 'Pranav', 90000),
- (104, 'Durga', 40000),
- (105, 'Raju', 60000),
- (106, 'Surya', 30000);

STEP 4: TO VERIFY DATA

- SELECT * FROM employees;
- Verified Data Output → Displays all the records of employees in the employees table

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including 'Databases (1)' and 'postgres'. The main pane shows a SQL query editor with the following code:

```

SELECT * FROM employees;

INSERT INTO employees (emp_id, emp_name, salary)
VALUES
(101, 'Priya', 25000),
(102, 'Jayaraj', 55000),
(103, 'Pranav', 90000),
(104, 'Durga', 40000),
(105, 'Raju', 60000),
(106, 'Surya', 30000);

SELECT * FROM employees;

```

The 'Data Output' pane shows the result of the query, displaying 6 rows of employee data:

emp_id	emp_name	salary
101	Priya	25000
102	Jayaraj	55000
103	Pranav	90000
104	Durga	40000
105	Raju	60000
106	Surya	30000

Total rows: 6. Query complete 00:00:00.582.

STEP 5: Apply CASE Statement

- ➔ SELECT
- ➔ emp_name,
- ➔ salary,
- ➔ CASE
- ➔ WHEN salary < 30000 THEN 'Low Salary'
- ➔ WHEN salary BETWEEN 30000 AND 60000 THEN 'Medium Salary'
- ➔ ELSE 'High Salary'
- ➔ END AS salary_category
- ➔ FROM employees;

STEP 5: Verified Data Output → Displays all the records of the employees salary categorised as Low, Medium and High

The screenshot shows the pgAdmin 4 interface with a SQL query editor containing the following code:

```

SELECT * FROM employees;

SELECT
emp_name,
salary,
CASE
WHEN salary < 30000 THEN 'Low Salary'
WHEN salary BETWEEN 30000 AND 60000 THEN 'Medium Salary'
ELSE 'High Salary'
END AS salary_category
FROM employees;

```

The 'Data Output' pane shows the result of the query, displaying 6 rows of employee data with their salary categories:

emp_name	salary	salary_category
Priya	25000	Low Salary
Jayaraj	55000	Medium Salary
Pranav	90000	High Salary
Durga	40000	Medium Salary
Raju	60000	Medium Salary
Surya	30000	Medium Salary

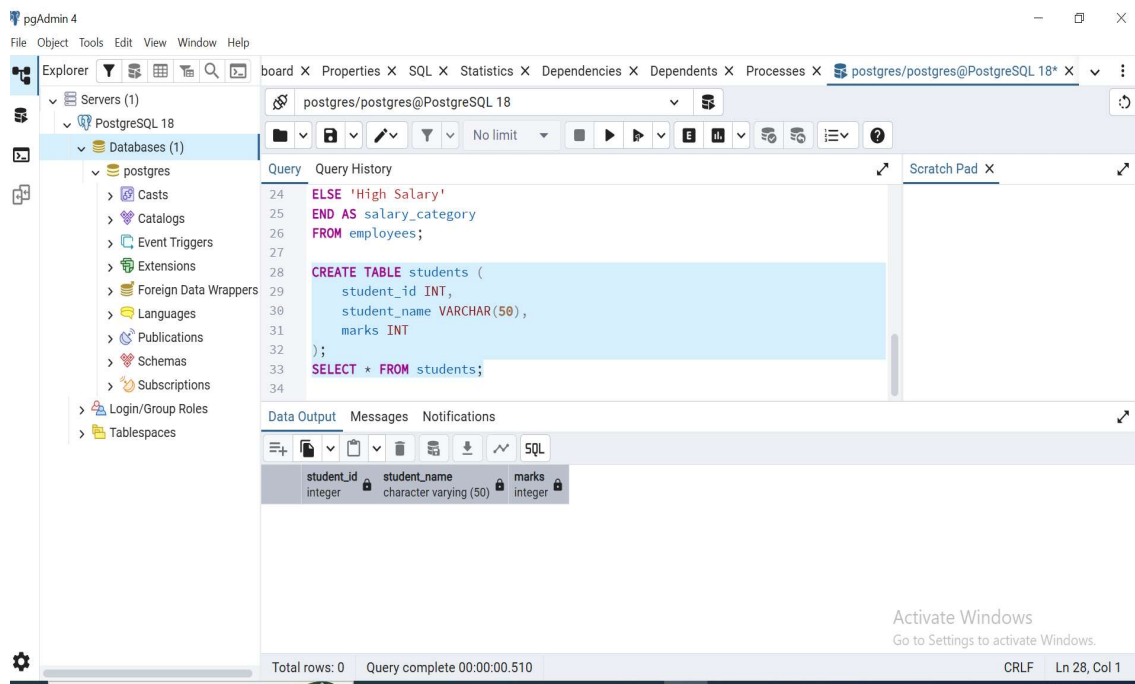
Total rows: 6. Query complete 00:00:00.429.

Another Example: Student Grades

Create Table

```
→ CREATE TABLE students (  
→   student_id INT,  
→   student_name VARCHAR(50),  
→   marks INT  
→ );  
→ SELECT * FROM students;
```

Verified Data Output → Displays the created student's table.



TO INSERT DATA

```
→ INSERT INTO students (student_id, student_name, marks)  
→ VALUES  
→ (1, 'Ravi', 95),  
→ (2, 'Priya', 78),  
→ (3, 'Kumar', 55),  
→ (4, 'Meena', 30),  
→ (5, 'Mohan', 65);
```

TO CHECK DATA

- ➔ **SELECT * FROM students;**
- ➔ **Verified Data output** ➔ Displays those records are entered in the student's table.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' and its 'Databases' list. The 'Query' tab is active, displaying a SQL script. The 'Data Output' tab shows the results of the query in a table format. The table has three columns: 'student_id' (integer), 'student_name' (character varying (50)), and 'marks' (integer). The data is as follows:

student_id	student_name	marks
1	Ravi	95
2	Priya	78
3	Kumar	55
4	Meena	30
5	Mohan	65

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.572'. A green message box at the bottom right says 'Successfully run. Total query runtime: 572 msec; 5 rows affected.'

CASE QUERY

- ➔ **SELECT**
- ➔ **student_name,**
- ➔ **marks,**
- ➔ **CASE**
- ➔ **WHEN marks >=90 THEN 'A Grade'**
- ➔ **WHEN marks >=60 THEN 'B Grade'**
- ➔ **WHEN marks >=40 THEN 'C Grade'**
- ➔ **ELSE 'Fail'**
- ➔ **END AS Grade**
- ➔ **FROM students;**

Verified Data Output → Display Student Grades → View students with grade classifications

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 18', and the 'Databases' tree is expanded to 'postgres'. The 'Query' tab is active, displaying the following SQL query:

```
43
44 SELECT
45 student_name,
46 marks,
47 CASE
48 WHEN marks >=90 THEN 'A Grade'
49 WHEN marks >=60 THEN 'B Grade'
50 WHEN marks >=40 THEN 'C Grade'
51 ELSE 'Fail'
52 END AS Grade
53 FROM students;
```

The 'Data Output' tab shows the results of the query in a table with 5 rows and 3 columns: student_name, marks, and grade. The data is as follows:

	student_name	marks	grade
1	Ravi	95	A Grade
2	Priya	78	B Grade
3	Kumar	55	C Grade
4	Meena	30	Fail
5	Mohan	65	B Grade

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.495'.

Real Interview Scenario: Orders

As the Orders Table already exists → DROP TABLE IF EXISTS orders;

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 18', and the 'Databases' tree is expanded to 'postgres'. The 'Query' tab is active, displaying the following SQL query:

```
55 CREATE TABLE orders(
56 order_id INT,
57 customer_name VARCHAR (50),
58 amount INT
59 );
60 SELECT * FROM orders;
61 DROP TABLE IF EXISTS orders;
62
63
64
65
```

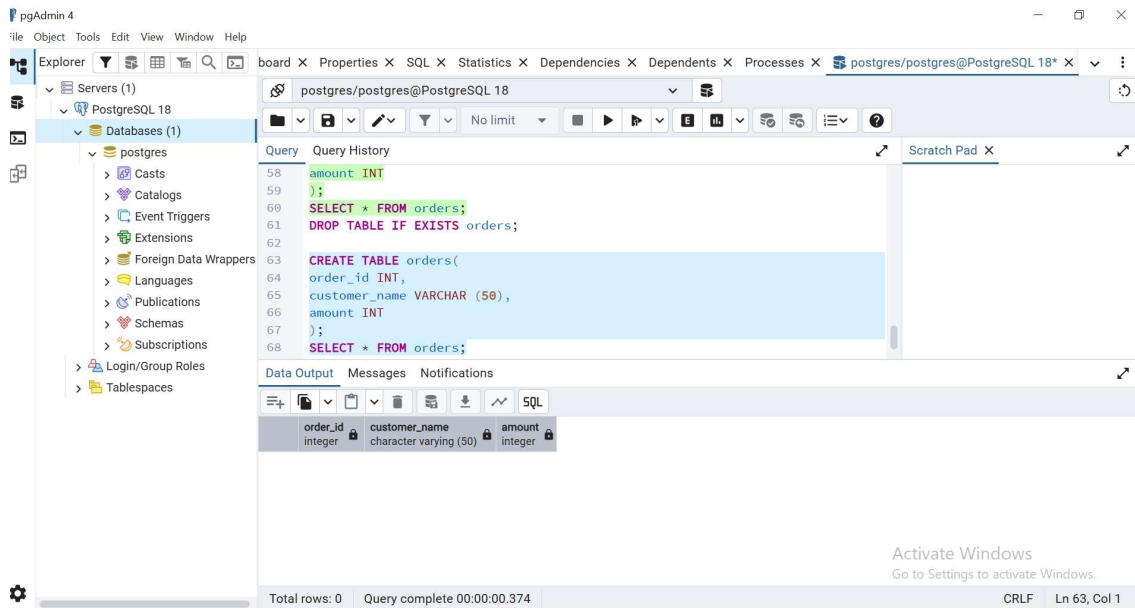
The 'Data Output' tab shows the results of the query in a table with 1 row and 1 column: DROP TABLE. The data is as follows:

	DROP TABLE
1	Query returned successfully in 391 msec.

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.391'.

Create Table

```
→ CREATE TABLE orders(  
→ order_id INT,  
→ customer_name VARCHAR (50),  
→ amount INT  
→ );  
→ SELECT * FROM orders;
```

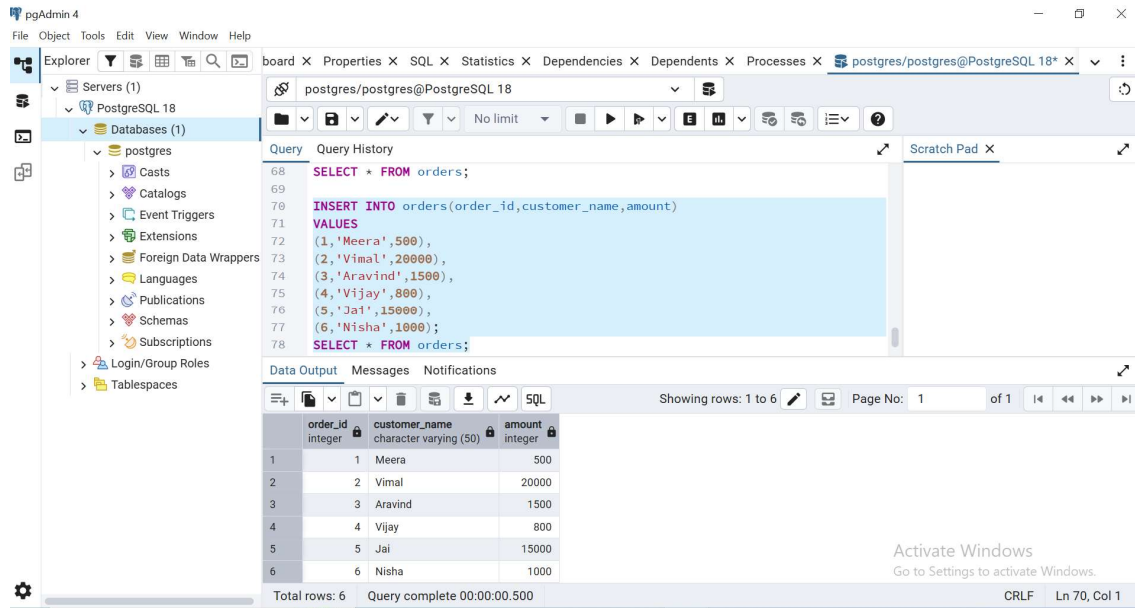


Insert Data

```
→ INSERT INTO orders(order_id,customer_name,amount)  
→ VALUES  
→ (1,'Meera',500),  
→ (2,'Vimal',20000),  
→ (3,'Aravind',1500),  
→ (4,'Vijay',800),  
→ (5,'Jai',15000),  
→ (6,'Nisha',1000);
```

Check Data

- ➔ **SELECT * FROM orders;**
- ➔ **Verify Order Data ➔ Display all order records using SELECT query.**



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show a PostgreSQL 18 server. Under 'Databases (1)', the 'postgres' database is selected. The 'Query' tab is active, showing the following SQL script:

```
68 SELECT * FROM orders;
69
70 INSERT INTO orders(order_id,customer_name,amount)
71 VALUES
72 (1, 'Meera', 500),
73 (2, 'Vimal', 20000),
74 (3, 'Aravind', 1500),
75 (4, 'Vijay', 800),
76 (5, 'Jai', 15000),
77 (6, 'Nisha', 1000);
78 SELECT * FROM orders;
```

The 'Data Output' tab shows the results of the SELECT query. The table has 6 rows and 3 columns: order_id, customer_name, and amount.

order_id	customer_name	amount
1	Meera	500
2	Vimal	20000
3	Aravind	1500
4	Vijay	800
5	Jai	15000
6	Nisha	1000

The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.500'.

CASE query

- ➔ **SELECT**
- ➔ **customer_name,**
- ➔ **amount,**
- ➔ **CASE**
- ➔ **WHEN amount < 1000 THEN 'Small Order'**
- ➔ **WHEN amount BETWEEN 1000 AND 10000 THEN 'Medium Order'**
- ➔ **ELSE 'Large Order'**
- ➔ **END AS order_category**
- ➔ **FROM orders;**

Display Spending Categories → View customers grouped by spending segment is categorised by LOW, MEDIUM and HIGH.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including 'Databases (1)' and 'postgres'. The main pane shows a SQL query in the 'Query' tab:

```

78 SELECT * FROM orders;
79
80 SELECT
81   customer_name,
82   amount,
83   CASE
84     WHEN amount < 1000 THEN 'Small Order'
85     WHEN amount BETWEEN 1000 AND 10000 THEN 'Medium Order'
86     ELSE 'Large Order'
87   END AS order_category
88 FROM orders;

```

The 'Data Output' tab shows the results of the query:

	customer_name character varying (50)	amount integer	order_category text
1	Meera	500	Small Order
2	Vimal	20000	Large Order
3	Aravind	1500	Medium Order
4	Vijay	800	Small Order
5	Jai	15000	Large Order
6	Nisha	1000	Medium Order

The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.480'.

SCENARIO 1 – MOVIE RATINGS

CREATE TABLE

- CREATE TABLE movies(
- movie_id INT,
- movie_name VARCHAR(100),
- rating DECIMAL(2,1)
-);

CHECK DATA

- SELECT * FROM movies;
- Verified Data Output Displays the Movie Table

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18'. The main pane shows a SQL query in the 'Query' tab:

```

86 ELSE 'Large Order'
87 END AS order_category
88 FROM orders;
89
90 CREATE TABLE movies(
91   movie_id INT,
92   movie_name VARCHAR(100),
93   rating DECIMAL(2,1)
94 );
95 SELECT * FROM movies;
96

```

The 'Data Output' tab shows the results of the query:

movie_id integer	movie_name character varying (100)	rating numeric (2,1)
---------------------	---------------------------------------	-------------------------

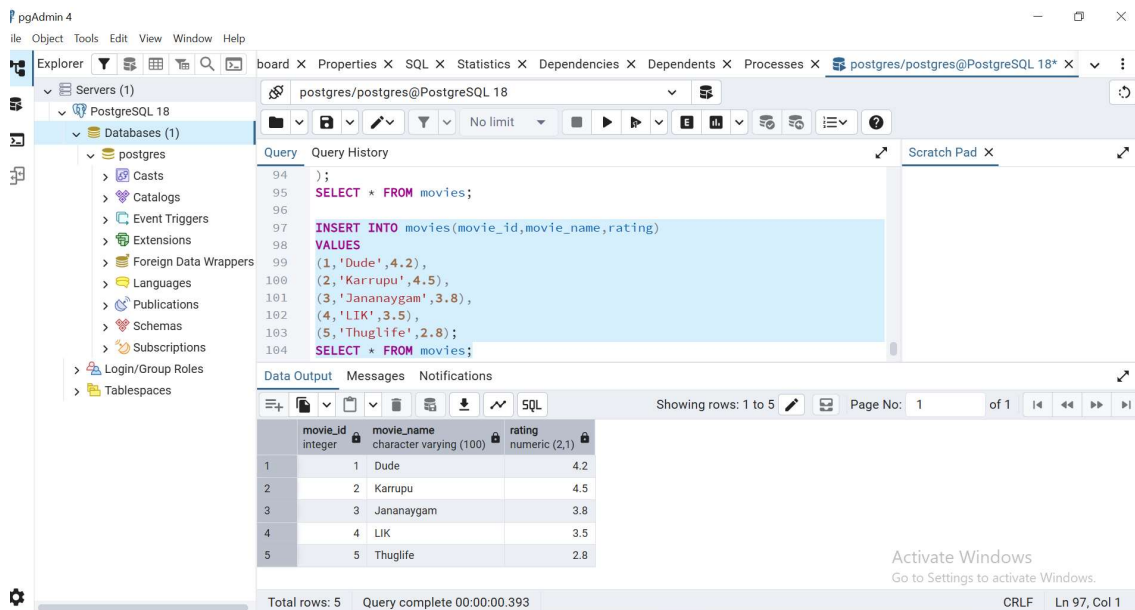
The status bar at the bottom indicates 'Total rows: 0' and 'Query complete 00:00:00.515'.

INSERT DATA

→ INSERT INTO movies(movie_id,movie_name,rating)
→ VALUES
→ (1,'Dude',4.2),
→ (2,'Karrupu',4.5),
→ (3,'Jananaygam',3.8),
→ (4,'LIK',3.5),
→ (5,'Thuglife',2.8);

CHECK DATA

→ SELECT * FROM movies;
→ Verified Data Output Displays the records inserted into the movies table.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'postgres'. The central pane shows the SQL query editor with the following code:

```
94 );  
95 SELECT * FROM movies;  
96  
97 INSERT INTO movies(movie_id,movie_name,rating)  
98 VALUES  
99 (1,'Dude',4.2),  
100 (2,'Karrupu',4.5),  
101 (3,'Jananaygam',3.8),  
102 (4,'LIK',3.5),  
103 (5,'Thuglife',2.8);  
104 SELECT * FROM movies;
```

The 'Data Output' pane at the bottom displays the results of the SELECT query as a table:

movie_id	movie_name	rating
1	Dude	4.2
2	Karrupu	4.5
3	Jananaygam	3.8
4	LIK	3.5
5	Thuglife	2.8

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.393'.

CASE QUERY

→ SELECT movie_name,rating,
→ CASE
→ WHEN rating >=4 THEN 'Excellent'
→ WHEN rating >=3 THEN 'Good'
→ ELSE 'Average'
→ END AS category

CHECK DATA

- ➔ FROM movies;
- ➔ Verified Data Output Displays the categorisation of the Movie ratings.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including 'Databases (1)' and 'postgres'. The main pane shows a SQL query in the 'Query' tab:

```
102 (4, 'LIK', 3.5),  
103 (5, 'Thuglife', 2.8);  
104 SELECT * FROM movies;  
105  
106 SELECT movie_name, rating,  
107 CASE  
108 WHEN rating >= 4 THEN 'Excellent'  
109 WHEN rating >= 3 THEN 'Good'  
110 ELSE 'Average'  
111 END AS category  
112 FROM movies;
```

The 'Data Output' tab shows the results of the query:

movie_name	rating	category
1	Dude	4.2
2	Karrupu	4.5
3	Jananaygam	3.8
4	LIK	3.5
5	Thuglife	2.8

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.558'.

USED GROUP BY and ORDER BY commands

- ➔ SELECT movie_name, rating
- ➔ FROM movies
- ➔ GROUP BY movie_name, rating
- ➔ ORDER BY rating DESC;
- ➔ Verified Data output shows the movie name and rating is grouped and the rating is arranged in the descending order.

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including 'Databases (1)' and 'postgres'. The main pane shows a SQL query in the 'Query' tab:

```
107 CASE  
108 WHEN rating >= 4 THEN 'Excellent'  
109 WHEN rating >= 3 THEN 'Good'  
110 ELSE 'Average'  
111 END AS category  
112 FROM movies;  
113  
114 SELECT movie_name, rating  
115 FROM movies  
116 GROUP BY movie_name, rating  
117 ORDER BY rating DESC;
```

The 'Data Output' tab shows the results of the query:

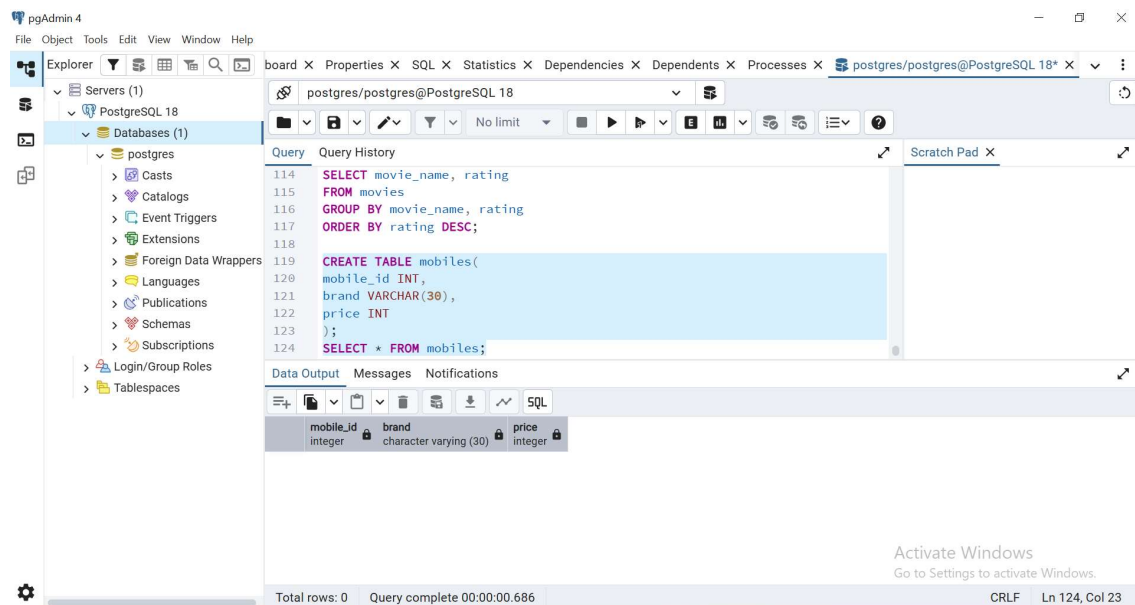
movie_name	rating
1	Karrupu
2	Dude
3	Jananaygam
4	LIK
5	Thuglife

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.427'.

SCENARIO 2 – MOBILE PHONES

CREATE DATE

- ➔ **CREATE TABLE mobiles(**
- ➔ **mobile_id INT,**
- ➔ **brand VARCHAR(30),**
- ➔ **price INT**
- ➔ **);**
- ➔ **SELECT * FROM mobiles;**
- ➔ **Verified Data Output displays the created table of mobiles.**



INSERT DATA

- ➔ **INSERT INTO mobiles VALUES**
- ➔ **(101,'Apple',120000),**
- ➔ **(102,'Samsung',70000),**
- ➔ **(103,'Vivo',45000),**
- ➔ **(104,'OnePlus',18000);**

CHECK DATA

- ➔ **SELECT * FROM mobiles;**
- ➔ **Verified Data Output displayed the mobile records in the table.**

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including 'Databases (1)', 'postgres', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'Login/Group Roles', and 'Tablespaces'. The main pane shows a SQL query in the 'Query' tab:

```
121 brand VARCHAR(30),
122 price INT
123 );
124 SELECT * FROM mobiles;
125
126 INSERT INTO mobiles VALUES
127 (101, 'Apple', 120000),
128 (102, 'Samsung', 70000),
129 (103, 'Vivo', 45000),
130 (104, 'OnePlus', 18000);
131 SELECT * FROM mobiles;
```

The 'Data Output' tab shows the results of the query:

mobile_id	brand	price
1	Apple	120000
2	Samsung	70000
3	Vivo	45000
4	OnePlus	18000

The status bar at the bottom indicates 'Total rows: 4' and 'Query complete 00:00:00.475'.

CASE QUERY

- ➔ **SELECT brand,price,**
- ➔ **CASE**
- ➔ **WHEN price>100000 THEN 'Premium'**
- ➔ **WHEN price BETWEEN 30000 AND 100000 THEN 'Mid Range'**
- ➔ **ELSE 'Budget'**
- ➔ **END AS Category**

CHECK DATA

- ➔ **FROM mobiles;**
- ➔ **Verified Data Output displays the record is been categorised the price as Premium, Mid Range and Budget.**

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18'. The main pane shows a SQL query in the 'Query' tab:

```
129 (103, 'Vivo', 45000),
130 (104, 'OnePlus', 18000);
131 SELECT * FROM mobiles;
132
133 SELECT brand,price,
134 CASE
135 WHEN price>100000 THEN 'Premium'
136 WHEN price BETWEEN 30000 AND 100000 THEN 'Mid Range'
137 ELSE 'Budget'
138 END AS Category
139 FROM mobiles;
```

The 'Data Output' tab shows the results of the query:

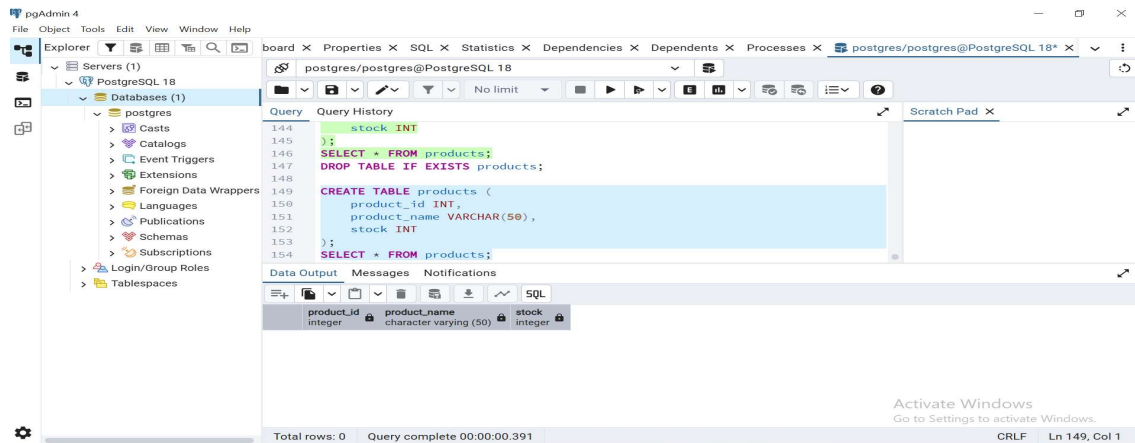
brand	price	category
Apple	120000	Premium
Samsung	70000	Mid Range
Vivo	45000	Mid Range
OnePlus	18000	Budget

The status bar at the bottom indicates 'Total rows: 4' and 'Query complete 00:00:00.516'.

Scenario 3: Amazon Products

Create Table

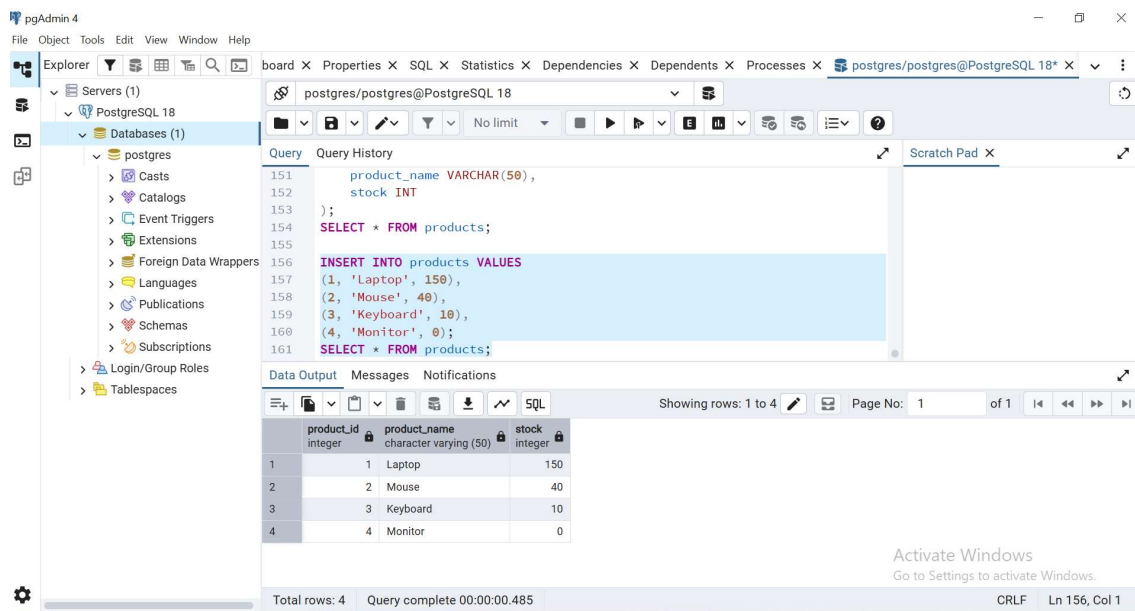
```
CREATE TABLE products (  
    product_id INT,  
    product_name VARCHAR(50),  
    stock INT  
);
```



Insert Data

```
INSERT INTO products VALUES  
(1, 'Laptop', 150),  
(2, 'Mouse', 40),  
(3, 'Keyboard', 10),  
(4, 'Monitor', 0);
```

```
SELECT * FROM products;
```



CASE Query

```
SELECT product_name,  
       stock,  
       CASE  
         WHEN stock > 100 THEN 'High Stock'  
         WHEN stock BETWEEN 20 AND 100 THEN 'Medium Stock'  
         ELSE 'Low Stock'  
       END AS inventory_status  
FROM products;
```

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' and its 'Databases' (postgres). The 'Query' tab is active, displaying the following SQL query:

```
SELECT product_name,stock,  
CASE  
WHEN stock >100 THEN 'High Stock'  
WHEN stock BETWEEN 20 AND 100 THEN 'Medium Stock'  
ELSE 'Low Stock'  
END AS inventory_status  
FROM products;
```

The 'Data Output' tab shows the results of the query in a table with 4 rows and 3 columns: product_name, stock, and inventory_status.

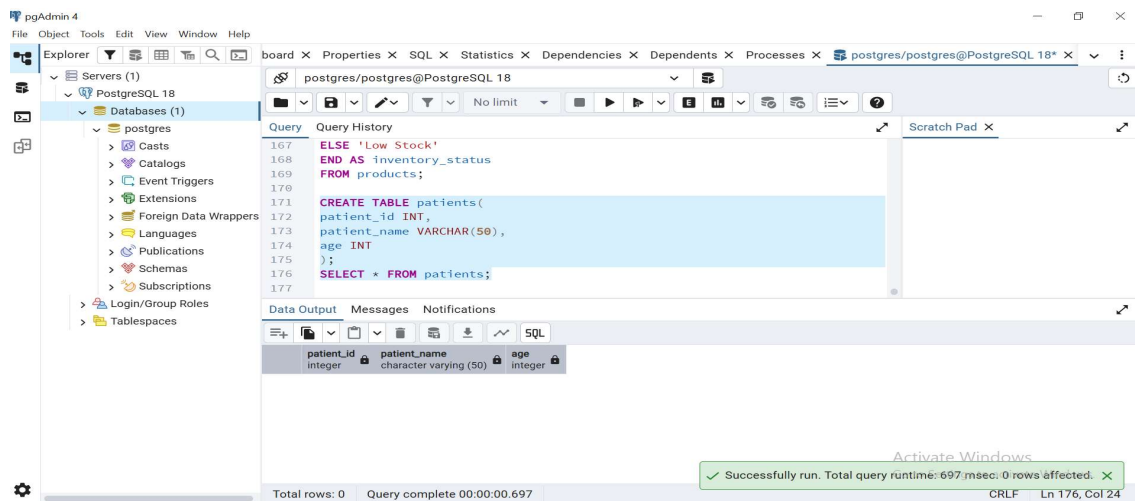
	product_name character varying (50)	stock integer	inventory_status text
1	Laptop	150	High Stock
2	Mouse	40	Medium Stock
3	Keyboard	10	Low Stock
4	Monitor	0	Low Stock

The status bar at the bottom indicates 'Total rows: 4' and 'Query complete 00:00:00.569'. The status bar also shows 'CRLF' and 'Ln 163, Col 1'.

Scenario 4: Hospital Patients

Create Table

```
CREATE TABLE patients (  
    patient_id INT,  
    patient_name VARCHAR(50),  
    age INT  
);
```



Insert Data

```
INSERT INTO patients VALUES
```

```
(1,'Vinoth',40),
```

```
(2,'Priya',30),
```

```
(3,'Adhitya',6),
```

```
(4,'Vijaya',63),
```

```
(5,'Ganesan',76),
```

```
(6,'Rithika',13);
```

```
SELECT * FROM patients;
```

pgAdmin 4

File Object Tools Edit View Window Help

board Properties SQL Statistics Dependencies Dependents Processes postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

Query Query History

```
175 );
176 SELECT * FROM patients;
177
178 INSERT INTO patients VALUES
179 (1,'Vinoth',40),
180 (2,'Priya',30),
181 (3,'Adhitya',6),
182 (4,'Vijaya',63),
183 (5,'Ganesan',76),
184 (6,'Rithika',13);
185 SELECT * FROM patients;
```

Data Output Messages Notifications

patient_id	patient_name	age
integer	character varying (50)	integer
1	Vinoth	40
2	Priya	30
3	Adhitya	6
4	Vijaya	63
5	Ganesan	76
6	Rithika	13

Total rows: 6 Query complete 00:00:00.651

Activate Windows
Go to Settings to activate Windows.

CRLF Ln 178, Col 1

ORDERED BY AGE IN ASCENDING ORDER

SELECT patient_name,age

FROM patients

GROUP BY patient_name,age

ORDER BY age;

pgAdmin 4

File Object Tools Edit View Window Help

board Properties SQL Statistics Dependencies Dependents Processes postgres/postgres@PostgreSQL 18*

postgres/postgres@PostgreSQL 18

Query Query History

```
180 (2,'Priya',30),
181 (3,'Adhitya',6),
182 (4,'Vijaya',63),
183 (5,'Ganesan',76),
184 (6,'Rithika',13);
185 SELECT * FROM patients;
186
187 SELECT patient_name,age
188 FROM patients
189 GROUP BY patient_name,age
190 ORDER BY age;
```

Data Output Messages Notifications

patient_name	age
character varying (50)	integer
Adhitya	6
Rithika	13
Priya	30
Vinoth	40
Vijaya	63
Ganesan	76

Total rows: 6 Query complete 00:00:00.422

Activate Windows
Go to Settings to activate Windows.

CRLF Ln 187, Col 1

CASE QUERY

```
SELECT patient_name,age,  
  
CASE  
  
WHEN age <18 THEN 'Child'  
  
WHEN age BETWEEN 18 AND 50 THEN 'ADULT'  
  
ELSE 'Senior Citizen'  
  
END AS age_group  
  
FROM patients;
```

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' and its 'Databases' list. The 'Query' tab is active, displaying the following SQL query:

```
191  
192 SELECT * FROM patients  
193 WHERE age >= 40;  
194  
195 SELECT patient_name,age,  
196 CASE  
197 WHEN age <18 THEN 'Child'  
198 WHEN age BETWEEN 18 AND 50 THEN 'ADULT'  
199 ELSE 'Senior Citizen'  
200 END AS age_group  
201 FROM patients;
```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with 6 rows and 3 columns: patient_name, age, and age_group.

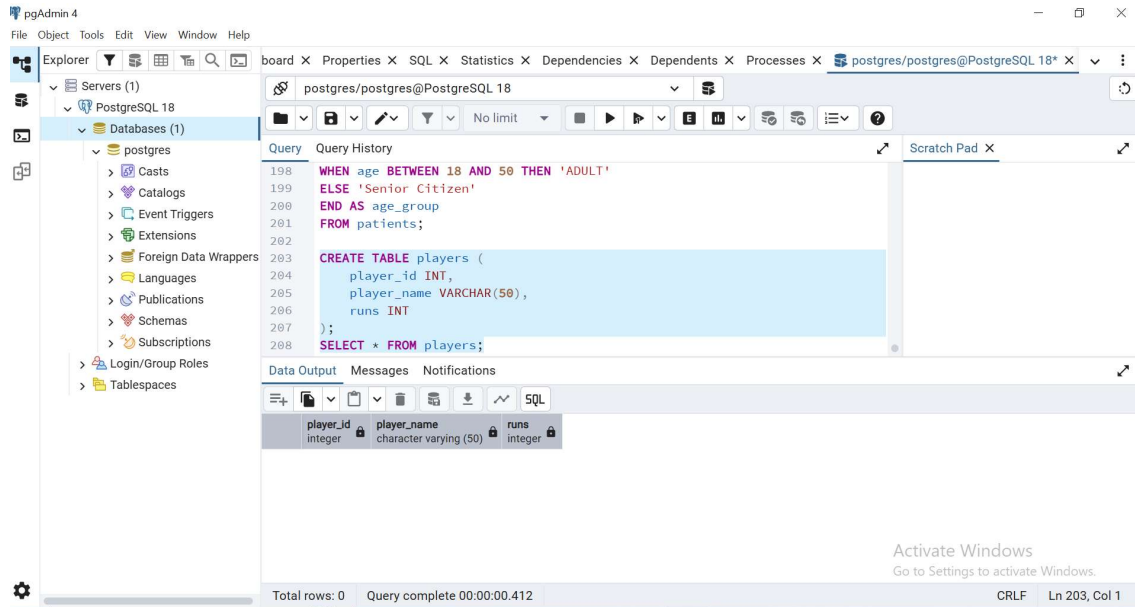
	patient_name	age	age_group
1	Vinoth	40	ADULT
2	Priya	30	ADULT
3	Adhitya	6	Child
4	Vijaya	63	Senior Citiz...
5	Ganesan	76	Senior Citiz...
6	Rithika	13	Child

The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.416'.

Scenario 5: Cricket Players

Create Table

```
CREATE TABLE players (  
    player_id INT,  
    player_name VARCHAR(50),  
    runs INT  
);  
  
SELECT * FROM players;
```



Insert Data

INSERT INTO players VALUES

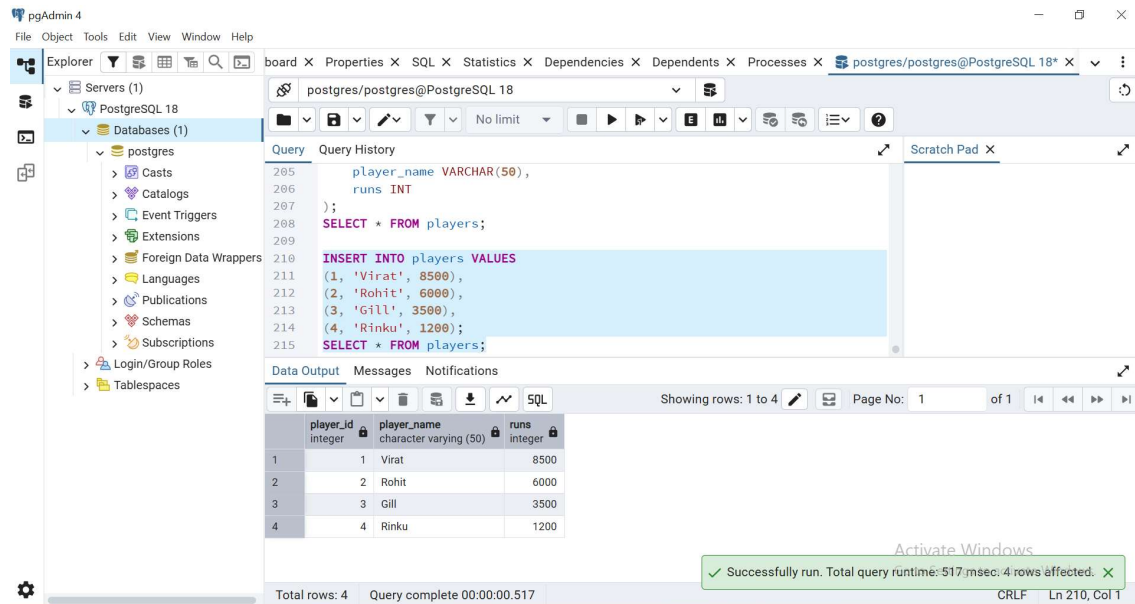
(1, 'Virat', 8500),

(2, 'Rohit', 6000),

(3, 'Gill', 3500),

(4, 'Rinku', 1200);

SELECT * FROM players;



CASE Query

SELECT player_name,runs,

CASE

WHEN runs >= 8000 THEN 'Legend'

WHEN runs >= 5000 THEN 'Senior Player'

ELSE 'Emerging Player'

END AS player_status

FROM players;

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' and its 'Databases' (postgres). The 'Query' editor is open, displaying the following SQL query:

```
(4, 'Rinku', 1200);  
SELECT * FROM players;  
  
SELECT player_name,runs,  
CASE  
WHEN runs >= 8000 THEN 'Legend'  
WHEN runs >= 5000 THEN 'Senior Player'  
ELSE 'Emerging Player'  
END AS player_status  
FROM players;
```

The 'Data Output' tab shows the results of the query in a table with 4 rows and 3 columns: player_name, runs, and player_status.

	player_name	runs	player_status
1	Virat	8500	Legend
2	Rohit	6000	Senior Player
3	Gill	3500	Emerging Play...
4	Rinku	1200	Emerging Play...

The status bar at the bottom indicates 'Total rows: 4' and 'Query complete 00:00:00.440'. The window title is 'postgres/postgres@PostgreSQL 18*'. An 'Activate Windows' watermark is visible in the bottom right corner.